1) Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.

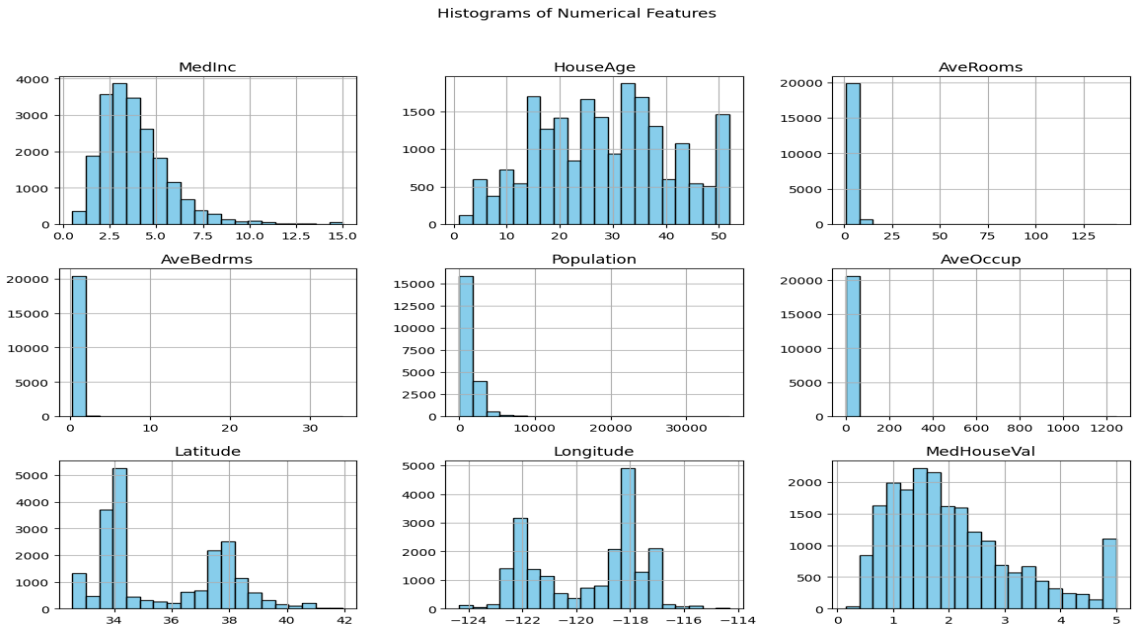
//Python program to create histograms and box plots for all numerical features in the California Housing dataset. This program uses the sklearn.datasets module to load the dataset and matplotlib and seaborn for visualization

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import fetch_california_housing

# Load California Housing dataset

data = fetch_california_housing(as_frame=True)

df = data.frame

# Display dataset information

print("Dataset Shape:", df.shape)

print("Dataset Columns:", df.columns)

# Histograms for numerical features

print("\nGenerating histograms...")

df.hist(bins=20, figsize=(14, 10), color='skyblue', edgecolor='black')

plt.suptitle('Histograms of Numerical Features', fontsize=16)

plt.show()

# Box plots for numerical features

print("\nGenerating box plots...")

plt.figure(figsize=(14, 10))

for i, col in enumerate(df.columns):

    plt.subplot(3, 3, i + 1)

    sns.boxplot(y=df[col], color='skyblue')

    plt.title(col)

    plt.tight_layout()

plt.show()
```

```
plt.suptitle('Box Plots of Numerical Features', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
# Analyze the outliers
print("\nOutlier Analysis:")
for col in df.columns:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"{col}: {len(outliers)} outliers")
```

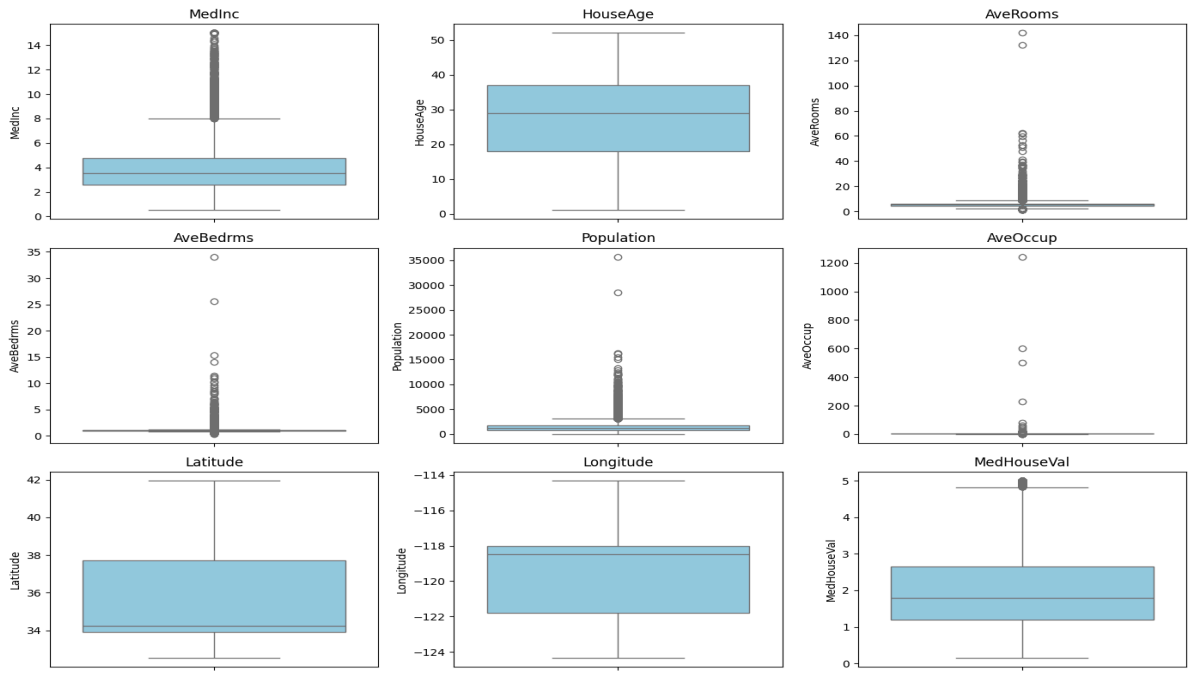
Output

```
Dataset Shape: (20640, 9)
Dataset Columns: Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
       'Latitude', 'Longitude', 'MedHouseVal'],
      dtype='object')
Generating histograms...
```



python introduction.docx

Generating box plots...



<Figure size 640x480 with 0 Axes>

Outlier Analysis:

MedInc: 681 outliers

HouseAge: 0 outliers

AveRooms: 511 outliers

AveBedrms: 1424 outliers

Population: 1196 outliers

AveOccup: 711 outliers

Latitude: 0 outliers

Longitude: 0 outliers

MedHouseVal: 1071 outliers

Program Explanation

Step 1: Import Required Libraries

Library	Purpose
pandas	To store data in table form (like Excel)
numpy	For numerical calculations
matplotlib	To draw graphs
seaborn	To draw statistical plots (box plots)
sklearn.datasets	To load built-in datasets

Step 2: Load the California Housing Dataset

```
data = fetch_california_housing(as_frame=True)
df = data.frame
```

Downloads the California Housing dataset

Converts it into a DataFrame (table format)

Stores it in variable df

Each row = one housing area

Each column = one numerical feature

Step 3: Display Dataset Information

```
print("Dataset Shape:", df.shape)
print("Dataset Columns:", df.columns)
```

Shows how big the dataset is

Displays feature names

OUTPUT – Dataset Information

Dataset Shape: (20640, 9)

Dataset Columns: Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms',

'Population', 'AveOccup', 'Latitude', 'Longitude', 'MedHouseVal'],

dtype='object')

Meaning of Output

20640 rows → housing records

9 columns → numerical features

Example features:

MedInc → Median income

HouseAge → Age of houses

MedHouseVal → Median house value

Step 4: Generate Histograms

A histogram is a graph that shows the distribution of data. It's a popular tool for analyzing large data sets, and can be used to identify outliers, gaps in data, and other features of a distribution. When to use a histogram

· Analyze data: Histograms are useful for analyzing data that's continuous or measured on an interval scale. For example, you can use a histogram to analyze test scores, customer satisfaction, or the number of people born in a certain decade.

· Identify outliers: Histograms can help you identify unusual observations, or outliers, in your data.

· Check for normality: Histograms can be used to check if your data is normally distributed.

· Compare processes: Histograms can help you compare the outputs of different processes.

· Communicate data: Histograms are a quick and easy way to communicate the distribution of data. How histograms work

· A histogram divides a data set into groups, or bins.

· For each group, a rectangle is constructed with a base length equal to the range of values in that group.

· The height of the rectangle is equal to the number of observations in that group.

· Histograms are similar to bar charts, but the bars touch each other.

```
df.hist(bins=20, figsize=(14, 10), color='skyblue', edgecolor='black')
plt.suptitle('Histograms of Numerical Features', fontsize=16)
```

```
plt.show()
```

What this step does

Creates histograms for all numerical features

Each histogram shows how values are distributed

Why histograms are used

To understand:

Data distribution

Skewness

Frequency of values

You will see **9 histograms** in one window.(OUTPUT)

Feature	Observation
MedInc	Right-skewed
AveRooms	Right-skewed
Population	Highly right-skewed
Latitude	Nearly symmetric
MedHouseVal	Right-skewed

Step 5: Generate Box Plots

A box plot, also known as a box and whisker plot, is a graph that summarizes data distribution. It's used to show the spread of numeric data, and to compare the distribution of data between different groups. Uses

· Compare data Box plots can be used to compare the distribution of data between multiple groups. For example, a market researcher might use a box plot to compare the annual revenue of different stores in a retail chain.

· Identify outliers Box plots can help identify outliers, which are data points that are far from the rest of the data. Outliers can indicate unusual occurrences or mistakes in the data.

· Check for normality Box plots can help determine if the data is normally distributed.

· Identify skewness Box plots can help identify the direction and magnitude of any asymmetry in the data.

· Analyze relationships Box plots can be used to analyze the relationship between a continuous feature and a categorical feature. What's included in a box plot?

· Median: The middle value of the data

· Quartiles: The upper and lower quartiles of the data

· Minimum and maximum: The smallest and largest values of the data

```
plt.figure(figsize=(14, 10))
for i, col in enumerate(df.columns):
plt.subplot(3, 3, i + 1)
sns.boxplot(y=df[col], color='skyblue')
plt.title(col)
```

What this step does

Draws one box plot for each feature

Arranges plots in a 3×3 grid

What a box plot shows

Median

Quartiles

Minimum & maximum

Outliers (dots outside the box)

OUTPUT – Box Plot Explanation

You will see **9 box plots**.

Typical Observations

Population → many outliers

AveOccup → extreme outliers

HouseAge → very few or no outliers

Latitude & Longitude → stable distribution

Dots outside the whiskers = outliers

Step 6: Outlier Detection Using IQR Method

```
q1 = df[col].quantile(0.25)
q3 = df[col].quantile(0.75)
iqr = q3 - q1
```

What this step does

Finds:Q1 → 25% value

Q3 → 75% value

IQR → spread of middle data

```
lower_bound = q1 - 1.5 * iqr
```

```
upper_bound = q3 + 1.5 * iqr
```

What this step does

Sets outlier limits

Values outside this range are considered outliers

```
outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
print(f"{col}: {len(outliers)} outliers")
```

What this step does

Counts outliers for each feature

Prints the result

Outlier Analysis:

MedInc: 681 outliers

HouseAge: 0 outliers

AveRooms: 511 outliers

AveBedrms: 1420 outliers

Population: 1196 outliers

AveOccup: 711 outliers

Latitude: 0 outliers

Longitude: 0 outliers

MedHouseVal: 1072 outliers

Meaning of Output

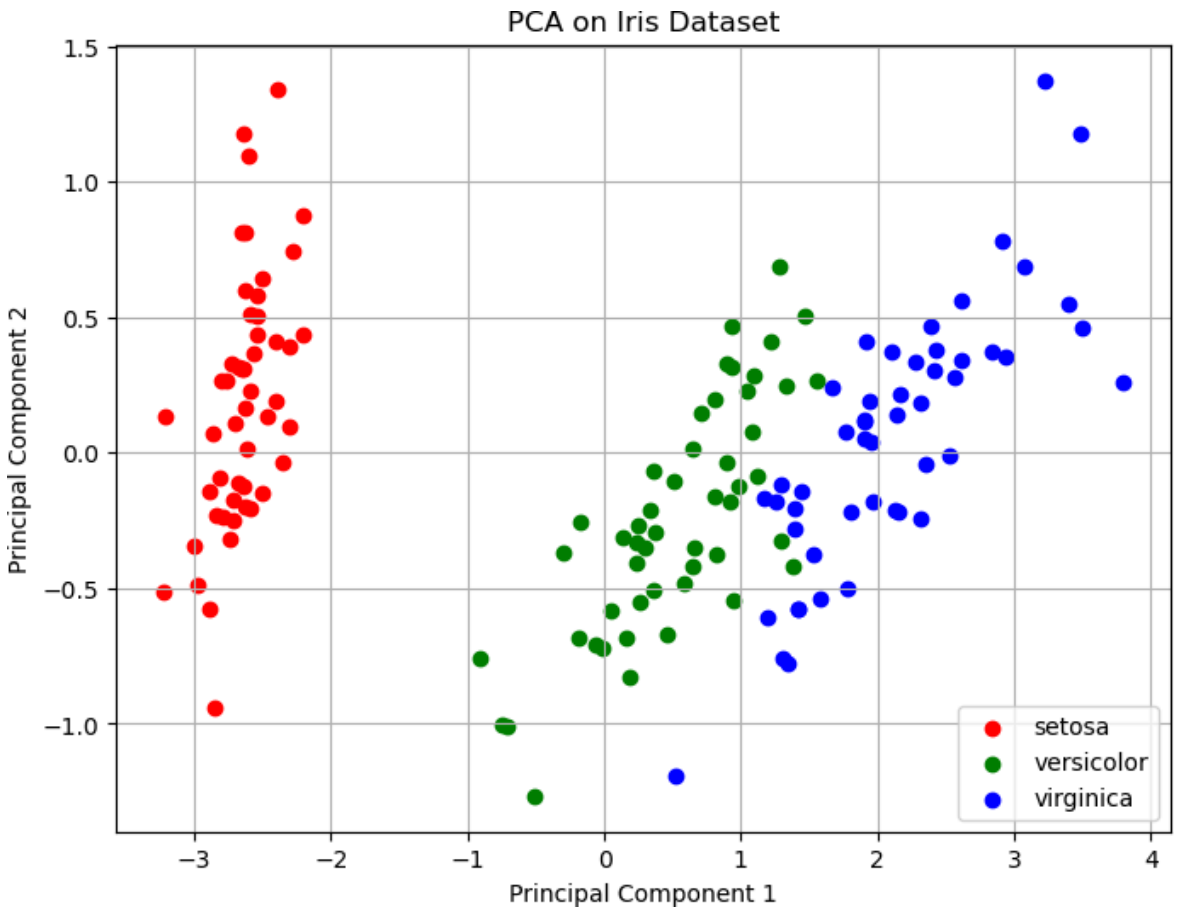
Features like Population have many outliers,Geographic features have very few outliers,

Economic data naturally contains extreme values

3.Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
label_names = iris.target_names

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Create DataFrame
df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df['Label'] = y

# Plot
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']

for i in np.unique(y):
    plt.scatter(
        df[df['Label'] == i]['PC1'],
        df[df['Label'] == i]['PC2'],
        color=colors[i],
        label=label_names[i]
    )

plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid()
plt.show()
```

output:



EXPLANATION:

1. **Import required libraries**

- Imports NumPy, used here for handling numerical operations (like finding unique labels).
- Imports Pandas, used to create and manage tabular data using DataFrames.
- Imports Matplotlib, a library used for plotting graphs and visualizations.
- Imports the Iris dataset from scikit-learn. This is a built-in dataset commonly used for machine learning examples.
- Imports PCA (Principal Component Analysis), a dimensionality-reduction technique.

---

2. **Load the dataset**

1. iris = load_iris()

Loads the Iris dataset into a dictionary-like object called iris.

2. X = iris.data

Extracts the feature matrix (150 samples × 4 features: sepal length, sepal width, petal length, petal width).

3. y = iris.target

Extracts the class labels:

0 → Setosa

1 → Versicolor

2 → Virginica

label_names = iris.target_names

Stores the actual class names corresponding to the numeric labels.

---

3. Apply PCA

- pca = PCA(n_components=2)

  Creates a PCA object that will reduce the data from 4 dimensions to 2 dimensions.

- X_pca = pca.fit_transform(X)

fit → learns how to project the data

transform → applies the projection

Result: X_pca contains the data in 2 principal components

---

4. Create a DataFrame

- df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

  Creates a DataFrame with:

PC1 → First principal component

PC2 → Second principal component

- df['Label'] = y

Adds the class labels to the DataFrame.

---

5. **Prepare the plot**

1. plt.figure(figsize=(8, 6))

   Creates a figure window of width 8 inches and height 6 inches.

2. colors = ['r', 'g', 'b']

   Defines colors for the three flower classes:

Red → Setosa

Green → Versicolor

Blue → Virginica

Plot each class separately

3. for i in np.unique(y):

Loops over the unique class labels (0, 1, 2).

```
plt.scatter(
 df[df['Label'] == i]['PC1'],
  df[df['Label'] == i]['PC2'],
 color=colors[i],
 label=label_names[i]
)
```

Selects only data points of class i

Plots PC1 vs PC2

Uses a different color and label for each class

---

6. **Add plot details**

4. plt.title('PCA on Iris Dataset')

   Sets the title of the graph.

5. plt.xlabel('Principal Component 1')

   Labels the x-axis.

6. plt.ylabel('Principal Component 2')

   Labels the y-axis.

7. plt.legend()

   Displays a legend identifying each flower class.

8. plt.grid()

   Adds a grid for better readability.

---

7. **Display the plot**

- plt.show()

  Shows the final PCA scatter plot.

What this program does overall

- Reduces a 4-feature dataset to 2 features using PCA
- Visualizes the data in 2D
- Clearly separates the three Iris flower classes

VIVA QUESTIONS Related lab program2

1. What is the Iris dataset?

Answer:

The Iris dataset is a famous dataset introduced by Ronald Fisher. It contains 150 samples of iris flowers with 4 features:

1. Sepal length

2. Sepal width
3. Petal length
4. Petal width

   It has 3 classes: Setosa, Versicolor, and Virginica.

---

2. What is PCA?

Answer:

PCA (Principal Component Analysis) is a dimensionality reduction technique that transforms high-dimensional data into lower dimensions while preserving maximum variance.

---

3. Why do we use PCA in this program?

Answer:

The Iris dataset has 4 features. PCA reduces it to 2 principal components so we can visualize it in a 2D plot.

---

4. What does n_components=2 mean?

Answer:

It means PCA will reduce the dataset to 2 principal components (2 new features).

---

5. What are principal components?

Answer:

Principal components are new variables that:

1. Are linear combinations of original features
2. Are orthogonal (uncorrelated)
3. Capture maximum variance

🔹 **Code-Based Questions**

6. What does pca.fit_transform(X) do?

Answer:

- fit() computes eigenvectors and eigenvalues
- transform() projects data onto principal components
- fit_transform() does both steps together

---

7. Why do we use np.unique(y) in the loop?

Answer:

To get unique class labels (0, 1, 2) so that each class can be plotted separately.

---

8. What is stored in X_pca?

Answer:

It contains the transformed data with 2 principal components instead of the original 4 features.

---

9. Why do we create a DataFrame after PCA?

Answer:

To:

- Easily manage data
- Add labels
- Filter data class-wise for plotting

---

10. Why are different colors used in the scatter plot?

Answer:

To visually distinguish the three iris species.

🔹 **Conceptual Questions**

11. Does PCA use class labels?

Answer:

No. PCA is an unsupervised technique. It does not use y values while computing components.

---

12. What does variance mean in PCA?

Answer:

Variance measures how spread out the data is. PCA keeps directions with maximum variance.

---

13. What happens if we use n_components=1?

Answer:

The dataset will be reduced to only 1 principal component, losing more information compared to 2 components.

---

14. What is the mathematical basis of PCA?

Answer:

PCA is based on:

3. Covariance matrix
4. Eigenvalues

5. Eigenvectors

---

15. What is explained variance ratio?

Answer:

It tells how much variance each principal component captures from the original dataset.

Example:

pca.explained_variance_ratio_

🔹 **Advanced Questions**

16. Why is PCA sensitive to scaling?

Answer:

Because PCA depends on variance. Features with larger scale dominate the principal components. So standardization is usually recommended.

---

17. How can you standardize the data before PCA?

Answer:

```
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
```

---

18. What is the shape of X before and after PCA?

Answer:

Before PCA: (150, 4)

After PCA: (150, 2)

---

19. Is PCA a feature selection or feature extraction method?

Answer:

Feature extraction, because it creates new features instead of selecting existing ones.

---

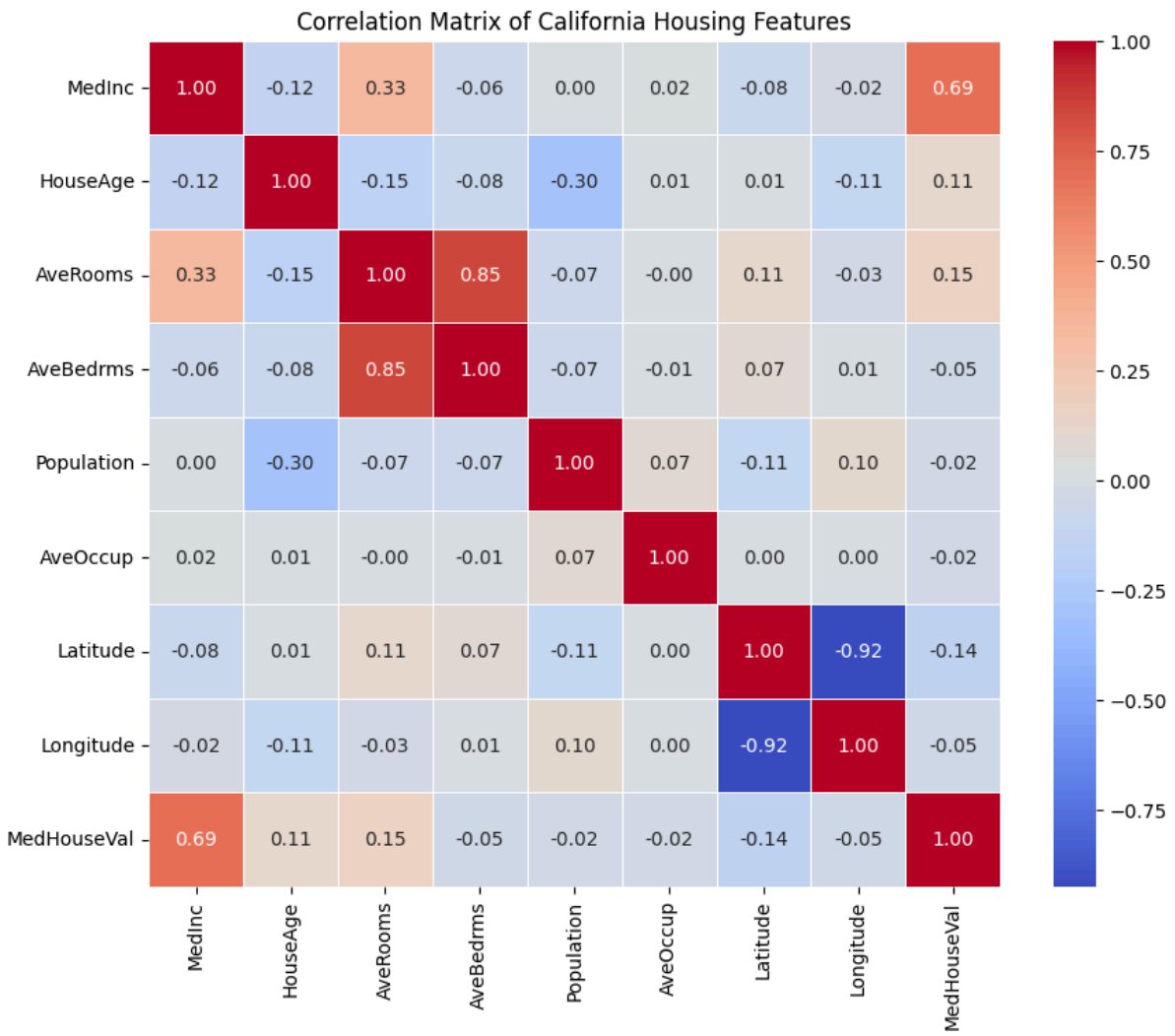
20. Why is Setosa clearly separated in PCA plot?

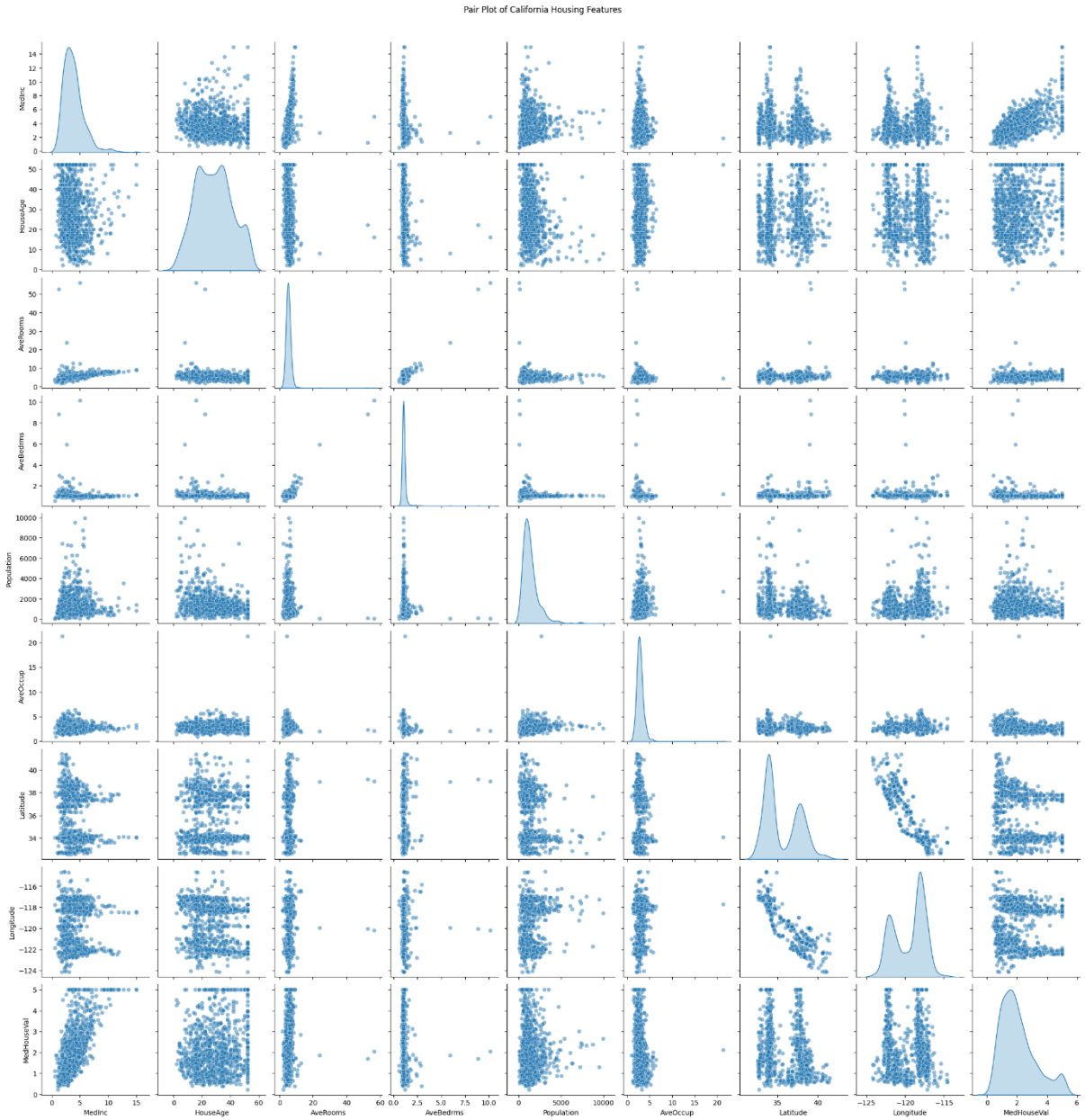
Answer:

Because Setosa has distinct petal and sepal measurements compared to the other two species.

2) Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

```
import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing


# Load dataset

df = fetch_california_housing(as_frame=True).frame


# Correlation heatmap

plt.figure(figsize=(10, 8))

sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)

plt.title('Correlation Matrix of California Housing Features')

plt.show()


# Pair plot (use sample for speed)

sns.pairplot(df.sample(1000), diag_kind='kde', plot_kws={'alpha': 0.5})

plt.suptitle('Pair Plot of California Housing Features', y=1.02)

plt.show()
```


Correlation Matrix of California Housing Features
MedInc
HouseAge
AveRooms
AveBedrms
Population
AveOccup
Latitude
Longitude
MedHouseVal
1.00 -0.12 0.33 -0.06 0.00 0.02 -0.08 -0.02 0.69
-0.12 1.00 -0.15 -0.08 -0.30 0.01 0.01 -0.11 0.11
0.33 -0.15 1.00 0.85 -0.07 -0.00 0.11 -0.03 0.15
-0.06 -0.08 0.85 1.00 -0.07 -0.01 0.07 0.01 -0.05
0.00 -0.30 -0.07 -0.07 1.00 0.07 -0.11 0.10 -0.02
0.02 0.01 -0.00 -0.01 0.07 1.00 0.00 0.00 -0.02
-0.08 0.01 0.11 0.07 -0.11 0.00 1.00 -0.92 -0.14
-0.02 -0.11 -0.03 0.01 0.10 0.00 -0.92 1.00 -0.05
0.69 0.11 0.15 -0.05 -0.02 -0.02 -0.14 -0.05 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75



Explanation:

Step 1: Import Required Libraries

Library	Purpose
seaborn (sns)	To create statistical visualizations
matplotlib (plt)	To display graphs
sklearn.datasets	To load the California Housing dataset

Step 2: Load the California Housing Dataset

df = fetch_california_housing(as_frame=True).frame

Loads the California Housing dataset

Converts it into a DataFrame (table format)

Stores it in the variable df

Dataset Contains

20640 rows → housing records

9 numerical features, such as:

MedInc (Median Income)

HouseAge

Population

MedHouseVal

Step 3: Compute the Correlation Matrix

df.corr()

Computes the **correlation coefficient** between every pair of features

Values range from:

+1 → strong positive correlation

0 → no correlation

–1 → strong negative correlation

This helps understand **how features are related to each other**.

Step 4: Create the Correlation Heatmap

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Matrix of California Housing Features')
plt.show()
```

Draws a **heatmap** to visually represent the correlation matrix

annot=True → displays correlation values inside the cells

cmap='coolwarm':

Red → strong positive correlation

Blue → strong negative correlation

fmt='.2f' → shows values up to 2 decimal places

OUTPUT

A colored grid showing the **strength and direction of relationships** between features.

Step 5: Create a Pair Plot

```
sns.pairplot(df.sample(1000), diag_kind='kde', plot_kws={'alpha': 0.5})
plt.suptitle('Pair Plot of California Housing Features', y=1.02)
plt.show()
```

Creates **scatter plots for each pair of features**

Diagonal plots show **data distribution (KDE)**

df.sample(1000) → uses a subset of data for faster plotting

alpha=0.5 → makes points slightly transparent

OUTPUT

A matrix of plots showing:

Pairwise relationships

Trends

Clusters

Outliers

Visualization	Purpose
Correlation Heatmap	Shows strength of linear relationships
Pair Plot	Shows detailed pairwise interactions

Explanation

1. Correlation Matrix:

o Computes the pairwise correlation between numerical features using df.corr().

o The correlation values range from -1 (strong negative) to 1 (strong positive), with 0 indicating no correlation.

2. Heatmap:

o Visualizes the correlation matrix using sns.heatmap(). The heatmap is color-coded for easier interpretation, with annotations showing correlation values.

3. Pair Plot:

o A pair plot is generated using sns.pairplot() to visualize pairwise relationships between features.

o It includes scatter plots for feature pairs and kernel density estimates (KDE) for diagonal plots.

Example Output

Correlation Matrix Heatmap

· Displays a grid where high positive or negative correlations are highlighted in red and blue, respectively.

Pair Plot

· Scatter plots and density plots for all feature pairs to visualize relationships, clustering, and potential outliers. Heatmap data visualization is a powerful tool used to represent numerical data graphically, where values are depicted using colors. This method is particularly effective for identifying patterns, trends, and anomalies within large datasets.

Why Heatmap Data Visualization?

Heatmaps offer a powerful way to visualize and analyze data, providing several advantages that make them a popular choice for data visualization:

· **Identifying Patterns and Trends:** Heatmaps allow for quick identification of patterns, trends, and correlations within datasets. By visually representing data with color gradients, heatmaps make it easier to discern areas of high or low concentration, enabling analysts to uncover insights that might not be apparent from raw data alone.

· **Handling Large Datasets:** Heatmaps are particularly effective for visualizing large datasets. They condense complex information into a compact, easily interpretable format, making it feasible to analyze and extract meaningful information from extensive data sets.

· **Intuitive Interpretation**: Heatmaps provide a visual representation of data that is intuitive and easy to interpret. The color gradients used in heatmaps offer a straightforward way to understand data density, intensity, and distribution, allowing viewers to grasp trends and patterns at a glance.

· **Comparing Data Sets:** Heatmaps enable side-by-side comparison of multiple datasets, facilitating the identification of similarities, differences, and relationships between variables. By overlaying or displaying multiple heatmaps simultaneously, analysts can gain insights into how different factors interact and influence each other.

· **Facilitating Decision Making**: Heatmaps help decision-makers make informed decisions by presenting complex data in a visually compelling format. Whether it's identifying areas of opportunity, allocating resources, or optimizing processes, heatmaps provide a clear and concise way to communicate insights and drive action.

· **Versatility:** Heatmaps can be applied across various domains and disciplines, including finance, healthcare, marketing, and science. From analyzing financial trends to tracking user behavior on websites, heatmaps offer a versatile tool for data visualization and analysis in diverse contexts.

· **Enhancing Communication:** Heatmaps are effective tools for communicating insights and findings to stakeholders, colleagues, and clients. Whether it's presenting research findings, reporting on key performance indicators, or visualizing spatial data, heatmaps help convey complex information in a visually appealing and accessible manner.

Types of Heatmaps

Heatmaps can be categorized based on the type of data they visualize and the specific use case. Here are some common types:

1. **Website Heatmaps**

Imagine you have a website, and you want to understand how visitors interact with it. A heatmap is like a map that shows you where visitors are spending the most time and where they're not. Think of it like this: the more time visitors spend on a particular section of your site, the "hotter" it gets on the heatmap. This is usually shown with warm colors like red or orange. So, if a section is red, it means it's getting a lot of attention.

Conversely, if a section is blue or green, it's "cooler," meaning visitors aren't spending much time there. So, blue or green areas indicate lower interaction.

Website heatmaps are used to visualize user behavior on web pages. They help identify which parts of a webpage receive the most interaction, such as clicks, scrolls, and mouse movements. · Click Maps: Show where users click on a webpage, helping to identify popular links and buttons.

· Scroll Maps: Indicate how far users scroll down a page, revealing which sections are most engaging.

· Mouse Tracking Heatmaps: Track mouse movements to understand which areas of a page attract the most attention.

· Eye-Tracking Heatmaps: Visualize where users' eyes focus on a page, providing insights into visual engagement. Website Heatmaps

2. **Grid Heatmaps**

Grid heatmaps are a powerful visualization tool used to represent data in a tabular format where each cell's color indicates the value of the data point it represents. This method is particularly effective for comparing multiple variables and identifying patterns, trends, and correlations within the data.

Key Characteristics:

1. **Color Encoding**: The primary feature of a heatmap is its use of color to represent data values. Colors are typically chosen from a gradient, with the intensity or hue indicating the magnitude of the values. For example, in a common color scheme, darker colors might represent higher values, while lighter colors represent lower values.

2. **Grid Layout:** The data is displayed in a grid, where each cell corresponds to a specific data point. The position of each cell is determined by its row and column, which often correspond to specific variables or categories.

3. **Data Density**: Heatmaps can handle large datasets effectively, making it easy to spot anomalies, trends, and clusters within the data. This is particularly useful in fields such as biology, where gene expression levels across different conditions can be visualized, or in finance, where stock price movements across different time periods are compared.

4. **Comparative Analysis:** By visualizing data in a grid format, heatmaps facilitate the comparison of multiple variables simultaneously. This can help identify correlations or relationships between different data points. Grid Heatmaps

3. Clustered Heatmaps

Clustered heatmaps extend the functionality of standard grid heatmaps by incorporating hierarchical clustering to show relationships between rows and columns. This added dimension of information makes clustered heatmaps particularly valuable in fields like biology, where they are commonly used to visualize genetic data.

Key Characteristics:

1. **Hierarchical Clustering:** Clustered heatmaps use hierarchical clustering algorithms to group similar rows and columns together. This clustering is often displayed as dendrograms (tree-like diagrams) alongside the heatmap, indicating the similarity between different rows or columns.

2**. Color Encoding**: As with standard heatmaps, the cell color represents the value of the data point. The color intensity or hue typically indicates the magnitude of the values, allowing for easy visual differentiation.

3. **Enhanced Patterns and Relationships:** By clustering similar rows and columns together, clustered heatmaps make it easier to identify patterns, correlations, and relationships within he data. This can reveal underlying structures that might not be immediately apparent in a standard heatmap.

4. **Interactive Exploration**: Many software tools and libraries allow users to interact with clustered heatmaps, enabling them to zoom in on specific clusters, reorder rows and columns, and explore the data in greater detail.

3. Clustered Heatmaps

Benefits of Heatmap Visualization Heatmaps offer several advantages over traditional data visualization methods:

· **Intuitive Understanding**: Colors make it easy to grasp complex data at a glance.

· **Pattern Recognition**: Heatmaps help identify patterns and trends that might be missed in numerical data.

· **Engagement**: The use of color makes heatmaps visually appealing and engaging.

· **Granularity:** Heatmaps provide detailed insights into data, allowing for more granular analysis.

When to Use Heatmap Visualization Heatmaps are versatile and can be used in various scenarios:

· **Website Optimization**: To understand user behavior and optimize webpage design.

· **Financial Analysis**: To visualize performance metrics and identify areas needing improvement.

· **Marketing**: To track campaign performance and customer engagement.

· **Scientific Research**: To analyze genetic data and other complex datasets.

· **Geographic Analysis**: Visualizing spatial data such as population density, crime rates, or weather patterns.

· **Sports Analytics**: Analyzing player movements, game strategies, or performance metrics.

Best Practices for Using Heatmaps for Data Visualization

To effectively use heatmaps, consider the following best practices:

· **Choose the Right Color Scale**: Selecting an appropriate color scale is crucial. Sequential scales are ideal for data that progresses in one direction, while diverging scales are suitable for data with a central neutral point and values that can be both positive and negative.

· **Ensure Sufficient Data**: Heatmaps require a large amount of data to be accurate. Analyzing heatmaps with insufficient data can lead to misleading conclusions.

· **Combine with Other Analytics**: Heatmaps should be used in conjunction with other analytics tools to provide a comprehensive understanding of the data. For example, combining heatmaps with form analytics can offer deeper insights into user behavior.

· **Use Legends**: Always include a legend to help interpret the color scale used in the heatmap. This ensures that viewers can accurately understand the data being presented.

· **Highlight Key Areas**: Use heatmaps to draw attention to important areas of the data. For example, in a website heatmap, highlight areas with the most user interaction to focus on optimizing those sections.

Different Tools for Generating Heatmaps

When it comes to generating heatmaps, several tools stand out for their features, ease of use, and effectiveness. Here are some of the best tools for generating heatmaps:

1. **VWO Insights**: VWO Insights is a comprehensive tool that offers advanced heatmap features. It provides predictive attention heatmaps, click maps, and scroll maps. VWO Insights is particularly useful for identifying user bottlenecks and optimizing website design. It integrates well with other tools for a holistic approach to user experience and conversion rate optimization.

2**. Hotjar**: Hotjar is a popular tool for creating website heatmaps. It supports click maps, scroll maps, and movement heatmaps. Hotjar also offers additional features like session recordings, feedback, surveys, and interviews, making it a robust tool for understanding user behavior and improving user experience. The free plan allows up to 35 daily sessions, and there are paid plans with more advanced features.

3. **SmartLook:** SmartLook provides three types of heatmaps: click maps, movement heatmaps, and scroll maps. It also offers session recordings, events, funnels, and crash reports. SmartLook is known for its cross-device viewing capabilities and integration with third-party testing solutions. However, the free version has limitations on sharing and downloading heatmaps.

4. **Microsoft Clarity**: Microsoft Clarity is a free tool that offers heatmaps along with session recordings and other analytics features. It is designed to help users understand how visitors interact with their website and identify areas for improvement.

5. **Google Analytics** (Page Analytics): Google Analytics offers a heatmap feature through its Chrome extension, Page Analytics. This tool provides a visual representation of where visitors click on a webpage, helping to identify popular and underperforming elements.

Conclusion

Heatmap data visualization is a powerful tool for representing complex data in an intuitive and engaging way. By using color to depict data values, heatmaps make it easier to identify patterns, trends, and anomalies. Whether used for website optimization, financial analysis, or scientific research, heatmaps provide valuable insights that can drive better decision-making

4.For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

```
import pandas as pd
def find_s_algorithm(file_path):
    data = pd.read_csv(file_path)
    print("Training data:")
    print(data)
    attributes = data.columns[:-1]  # All columns except last
    class_label = data.columns[-1]  # Last column is class label
    hypothesis = ['?' for _ in attributes]
    for index, row in data.iterrows():
        if row[class_label] == 'Yes':
            for i, value in enumerate(row[attributes]):
                if hypothesis[i] == '?' or hypothesis[i] == value:
                    hypothesis[i] = value
                else:
                    hypothesis[i] = '?'
    return hypothesis

# Define file path
file_path = r"D:\training_data.csv"

# Call the Find-S algorithm
hypothesis = find_s_algorithm(file_path)

# Print final hypothesis
print("\nThe final hypothesis is:", hypothesis)
```

Data Set:

```
Sky,AirTemp,Humidity,Wind,Water,Forecast,EnjoySport
Sunny,Warm,Normal,Strong,Warm,Same,Yes
Sunny,Warm,High,Strong,Warm,Same,Yes
Rainy,Cold,High,Strong,Warm,Change,No
Sunny,Warm,High,Strong,Cool,Change,Yes
```

Program Explanation:

Define the Function

```
def find_s_algorithm(file_path):
```

We are defining a function named find_s_algorithm.

It takes one argument → file_path

This path tells Python where your dataset is stored.

Read the Dataset

```
data = pd.read_csv(file_path)
```

Opens the CSV file, Converts it into a DataFrame (table format), Stores it in variable data

Print Training Data

```
print("Training data:")
print(data)
```

This just displays the dataset so you can verify it loaded correctly.

Identify Attributes and Class Label

```
attributes = data.columns[:-1]
class_label = data.columns[-1]
```

data.columns gives all column names.

[:-1] means: take all columns except last one.

[-1] means: take the last column.

5. attributes = input features (Sky, AirTemp, etc.)
6. class_label = target output (EnjoySport)

Initialize Hypothesis

```
hypothesis = ['?' for _ in attributes]
```

This creates a list of '?' symbols.

Example:

If there are 6 attributes:

['?', '?', '?', '?', '?', '?']

Loop Through Each Training Example

```
for index, row in data.iterrows():
```

iterrows() goes row by row. row represents one training example.

Check Only Positive Examples

```
if str(row[class_label]).strip().lower() == 'yes':
```

This means:

Only consider rows where output is Yes

Ignore negative examples (No)

Update Hypothesis

```
for i, value in enumerate(row[attributes]):
```

enumerate() gives index and value.

i = attribute position

value = actual value in that row

Case 1: If hypothesis is still '?

```
if hypothesis[i] == '?':
  hypothesis[i] = value
```

Replace '?' with that attribute value.

Case 2: If value differs

```
elif hypothesis[i] != value:
    hypothesis[i] = '?'
```

If values conflict across positive examples:

Generalize it again to '?'

Return Final Hypothesis

```
return hypothesis
```

After processing all rows, the final consistent hypothesis is returned.

Call Function

```
hypothesis = find_s_algorithm(file_path)
```

Print Final Result

```
print("\nFinal Hypothesis:")
print(hypothesis)
```

Displays learned hypothesis.

Find-S algorithm

The **Find-S algorithm** is a simple concept learning method that finds the *most specific hypothesis* consistent with all positive examples. It works best on **noise-free datasets** and ignores negative examples, making it ideal for educational purposes and small datasets.

Steps to Implement:

Load the dataset from a CSV file.

Initialize the hypothesis to the most specific form (e.g., ['0', '0', ...]).

Iterate through each training example: If the example is negative, skip it. If positive: Replace '0' with the attribute value. If values differ, replace with '?'.

Return the final hypothesis after processing all examples.

Hypothesis in machine Learning

In **machine learning**, a hypothesis is an **assumed mapping function** between input features and the target output. It represents the model's belief about how inputs relate to outputs, and the learning process involves adjusting parameters to find the hypothesis that best fits the data while generalizing well to unseen cases .

A hypothesis is **not the same as a model**—the hypothesis is the assumption, while the model is the mathematical representation used to test it.

Viva Questions

What is the Find-S algorithm?

Find-S is a supervised learning algorithm that finds the most specific hypothesis that fits all positive training examples.

What type of learning does Find-S use?

It uses **concept learning** in supervised learning.

What is the goal of this program?

To find the most specific hypothesis from the given training dataset stored in a CSV file.

Why do we use pandas in this program?

To read and handle the CSV dataset easily using pd.read_csv().

What does data.columns[:-1] mean?

It selects all columns except the last one (attributes only).

Why is data.columns[-1] used?

To select the last column as the class label.

- **Program Understanding Questions**

Why is the hypothesis initialized with '?'?

'?' represents the most general value for each attribute.

Why do we check if row[class_label] == 'Yes'?

Because Find-S considers only positive examples.

What happens if the attribute value matches the hypothesis?

The hypothesis remains unchanged.

What happens if two positive examples have different values for an attribute?

That attribute in the hypothesis becomes '?'.

What is the role of enumerate() in the program?

It provides both index and value while iterating over attributes.

What does data.iterrows() do?

It iterates through each row of the dataset.

🔹 Conceptual Questions

Does Find-S consider negative examples?

No, it ignores negative examples.

What is the limitation of Find-S?

Ignores negative examples

Assumes noise-free data

Finds only one hypothesis

What type of hypothesis space does Find-S work on?

Conjunctive hypothesis space.

Is Find-S guaranteed to find the correct concept?

Only if the training data is noise-free and consistent.

What is version space?

The set of all hypotheses consistent with training examples.

Which algorithm improves over Find-S?

Candidate Elimination Algorithm.

🔹 Output-Based Questions

What does the final hypothesis represent?

The most specific rule that classifies all positive examples correctly.

What will happen if there are no positive examples?

The hypothesis remains ['?', '?', ...].

What will happen if all examples are positive?

The hypothesis becomes the most specific matching rule.

🔹 **Practical Questions**

- **What should the CSV file contain?**

  8. Attribute columns
  9. Last column as class label (Yes/No)

- **Why is the file path written as r"D:\training_data2.csv"?**

  r makes it a raw string to avoid escape character errors.

- **What error occurs if the file path is wrong?**

  FileNotFoundError

- **How can this program be improved?**

  4. Add error handling
  5. Handle different class labels
  6. Make it case-insensitive

OUTPUT:

Training Data:
The Final Hypothesis is : ['?', 'Good', 'Yes', 'Easy']

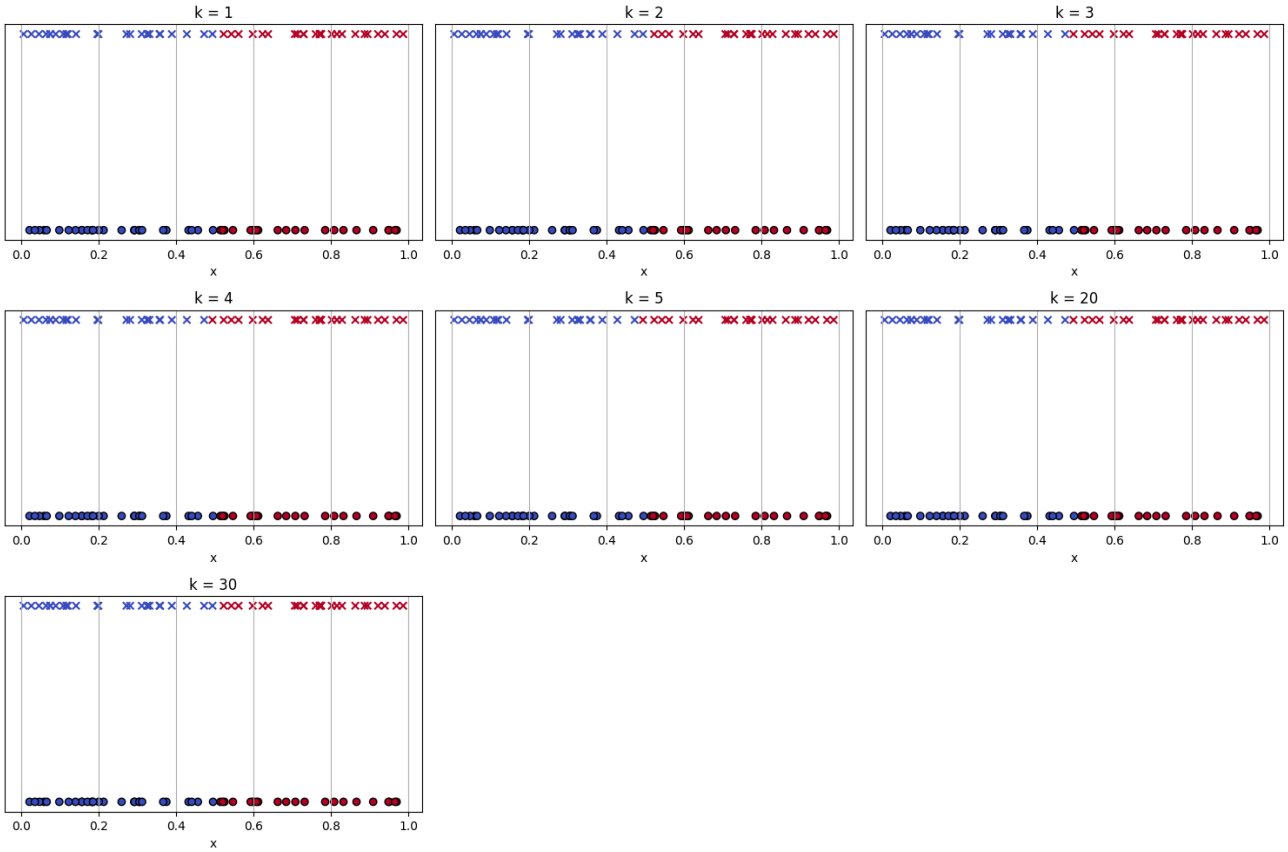
Study	Attendance	Assignment	Exam	Result
High	Good	Yes	Easy	Yes
High	Good	Yes	Hard	Yes
Medium	Good	Yes	Easy	Yes
Low	Poor	No	Easy	No
High	Poor	Yes	Easy	No

Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated. a. Label the first 50 points {x1,……,x50} as follows: if (xi ≤ 0.5), then xi ε Class1, else xi ε Class1 b. Classify the remaining points, x51,……,x100 using KNN. Perform this for k=1,2,3,4,5,20,30

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

np.random.seed(42)

X = np.random.rand(100,1)
y = (X[:50,0] > 0.5).astype(int)

ks = [1,2,3,4,5,20,30]

plt.figure(figsize=(15,10))

for i, k in enumerate(ks, 1):
    p = KNeighborsClassifier(k).fit(X[:50], y).predict(X[50:])
    print(f'k={k}:', p)

    plt.subplot(3,3,i)
    plt.scatter(X[:50], np.zeros(50), c=y, cmap='coolwarm', edgecolors='k')
    plt.scatter(X[50:], np.ones(50)*0.1, c=p, cmap='coolwarm', marker='x')

    plt.title(f"k = {k}")
```

```
    plt.yticks([])
    plt.xlabel('x')
    plt.grid()

plt.tight_layout()
plt.show()
```

```
k=1: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 0 1 0 0 0]
k=2: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 0 1 0 0 0]
k=3: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 1 1 0 0 0]
k=4: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 1 1 0 0 0]
k=5: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 1 1 0 0 0]
k=20: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 1 1 0 0 0]
k=30: [1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 0 0 0 0 1
 1 1 0 0 1 1 1 1 0 1 0 0 0]
```



Explanation Of Program

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
```

from sklearn.neighbors import KNeighborsClassifier

Imports the KNN classification algorithm.

2. Set Random Seed

np.random.seed(42)

This ensures that the same random numbers are generated every time the program runs.

Without this, the random dataset would change every time you run the program.

3. Generate Random Data

X = np.random.rand(100,1)

This generates 100 random numbers between 0 and 1. Shape of X: 100 data points, 1 feature

4. Create Class Labels

y = (X[:50,0] > 0.5).astype(int)

This creates labels for the first 50 data points**.**

Select first 50 points

X[:50]

Check condition

X[:50,0] > 0.5

Ifvalue>0.5→True
Else → False

Convert Boolean to integer:

.astype(int)

True→1
False → 0

So we get binary classes:

0=blue
1 = red

5. Choose Different K Values

ks = [1,2,3,4,5,20,30]

These are the **number of neighbors used in KNN**.

Small K → very sensitive model
Large K → smoother predictions

6. Create Plot Figure

```
plt.figure(figsize=(15,10))
```

Creates a large figure window for plotting.

Size = **15 × 10 inches**.

7. Loop Through Each K Value

```
for i, k in enumerate(ks, 1):
```

This loop runs **7 times**.

8. Train the KNN Model

```
p = KNeighborsClassifier(k).fit(X[:50], y).predict(X[50:])
```

Step 1: Create KNN Model

KNeighborsClassifier(k)

Creates a classifier with **k neighbors**.

Example:

k=3 → use 3 nearest neighbors

Step 2: Train Model

.fit(X[:50], y)

Training data:

X[:50]

Training labels:

y

The model stores the training points.

KNN does **not actually learn parameters**.

It simply **stores the dataset**.

Step 3: Predict New Points

.predict(X[50:])

Testing data:

X[50:]

These are the **remaining 50 data points**.

For each test point:

- Calculate distance to training points
- Select k nearest neighbors
- Take majority class
- Assign predicted class

Predicted labels are stored in:

p

9. Print Predictions

```
print(f ' k={k}:', p)
```

Example output:

```
k=1: [0 0 1 1 0 ...]
k=5: [0 0 0 1 1 ...]
```

This shows predicted classes for each value of k.

10. Create Subplot

```
plt.subplot(3,3,i)
```

Creates a **3 × 3 grid of graphs**.

Each graph corresponds to one **k value**.

11. Plot Training Data

```
plt.scatter(X[:50], np.zeros(50), c=y, cmap='coolwarm', edgecolors='k')
```

Training points are plotted at:

y = 0

Color represents class:

Color	Class
Blue	0
Red	1

12. Plot Predicted Data

```
plt.scatter(X[50:], np.ones(50)*0.1, c=p, cmap='coolwarm', marker='x')
```

Test points plotted slightly above training points.

y = 0.1

Marker:

x

Represents predicted class.

13. Add Title

```
plt.title(f " k = {k}")
```

Displays the value of **k** on each graph.

14. Remove Y Axis

```
plt.yticks([])
```

Because the graph only shows **x distribution**, the y-axis is not needed.

15. Label X Axis

```
plt.xlabel('x')
```

Shows the feature variable.

16. Add Grid

```
plt.grid()
```

Makes the plot easier to read.

17. Adjust Layout

```
plt.tight_layout()
```

Prevents overlapping graphs.

18. Show Plot

```
plt.show()
```

Displays the visualization.

What the Visualization Shows

The graphs illustrate how **changing k affects classification**.

Case 1: Small K (k=1)

Model looks at only **1 neighbor**.

Very sensitive to noise.

This can cause **overfitting**.

Case 2: Medium K (k=3 or 5)

More stable classification.

Balances bias and variance.

Case 3: Large K (k=20 or 30)

Many neighbors considered.

Prediction becomes smoother.

May cause **underfitting**.

This program demonstrates how **KNN decision boundaries change when the number of neighbors changes**.

It visually shows:

4. Overfitting (small k)
5. Balanced model (medium k)
6. Underfitting (large k)

K-Nearest Neighbors (KNN) Algorithm – Conceptual Explanation

1. Introduction

K-Nearest Neighbors (KNN) is a **supervised machine learning algorithm** used for both **classification and regression** problems. It belongs to a class of algorithms called **instance-based learning** or **lazy learning**, because it does not build an explicit model during the training phase. Instead, it **stores the training data** and makes predictions when a new data point is presented.

The fundamental assumption of KNN is that **data points that are close to each other in the feature space are likely to belong to the same class**. Therefore, the class of a new sample can be determined by examining the classes of its nearest neighbors.

2. Basic Idea of KNN

Suppose we have a dataset with labeled examples. When a new data point arrives, the algorithm:

7. Calculates the **distance** between the new data point and all existing training data points.
8. Identifies the **K closest points** (neighbors).
9. Uses **majority voting** among those neighbors to determine the class of the new data point.

3. Step-by-Step Algorithm

The KNN algorithm follows these steps:

8. Choose a value for **K** (number of neighbors).
9. Compute the **distance** between the new data point and all training samples.
10. Sort the distances in **ascending order**.
11. Select the **K nearest neighbors**.
12. Determine the **majority class** among those neighbors.
13. Assign that class as the **prediction**.

4. Mathematical Representation

Let the training dataset be:

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

Where:

7. x_i= feature vector
8. y_i= class label

For a new point x_q, the distance to every training point is calculated.

The most commonly used distance is **Euclidean Distance**:

$$d(x_i, x_q) = \sqrt{\sum_{j=1}^{m} (x_{ij} - x_{qj})^2}$$

Where:

- m= number of features.

After computing all distances, the algorithm selects the **K smallest distances**.

5. Example of KNN Classification

Consider the following dataset:

Feature Value	Class
0.1	0
0.2	0
0.3	0
0.6	1
0.8	1

Suppose a new data point appears:

$$x = 0.65$$

Distances from training points:

Training Point	Distance	Class
0.6	0.05	1
0.8	0.15	1
0.3	0.35	0

If **K = 3**

Neighbors:

1, 1, 0

Majority class = **1**

Prediction = **Class 1**

6. Distance Measures in KNN

Different distance metrics can be used depending on the dataset.

Euclidean Distance

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Manhattan Distance

$$d = |x_1 - y_1| + |x_2 - y_2|$$

Minkowski Distance

$$d = (|x_1 - y_1|^p + |x_2 - y_2|^p)^{1/p}$$

7. Choosing the Value of K

The parameter **K** plays a crucial role in the performance of the algorithm.

Value of K	**Effect**
Small K	Sensitive to noise (overfitting)
Large K	Smooth decision boundary (underfitting)

Usually **odd values of K** are chosen for binary classification to avoid ties.

8. Advantages of KNN

- Simple and easy to understand
- No training phase required
- Works well for small datasets
- Can capture nonlinear patterns

9. Disadvantages of KNN

3. Slow for large datasets because distance must be computed for every point
4. Sensitive to irrelevant features
5. Requires feature scaling
6. High memory requirement since all data must be stored

10. Time Complexity

Training complexity:

$$O(1)$$

Prediction complexity:

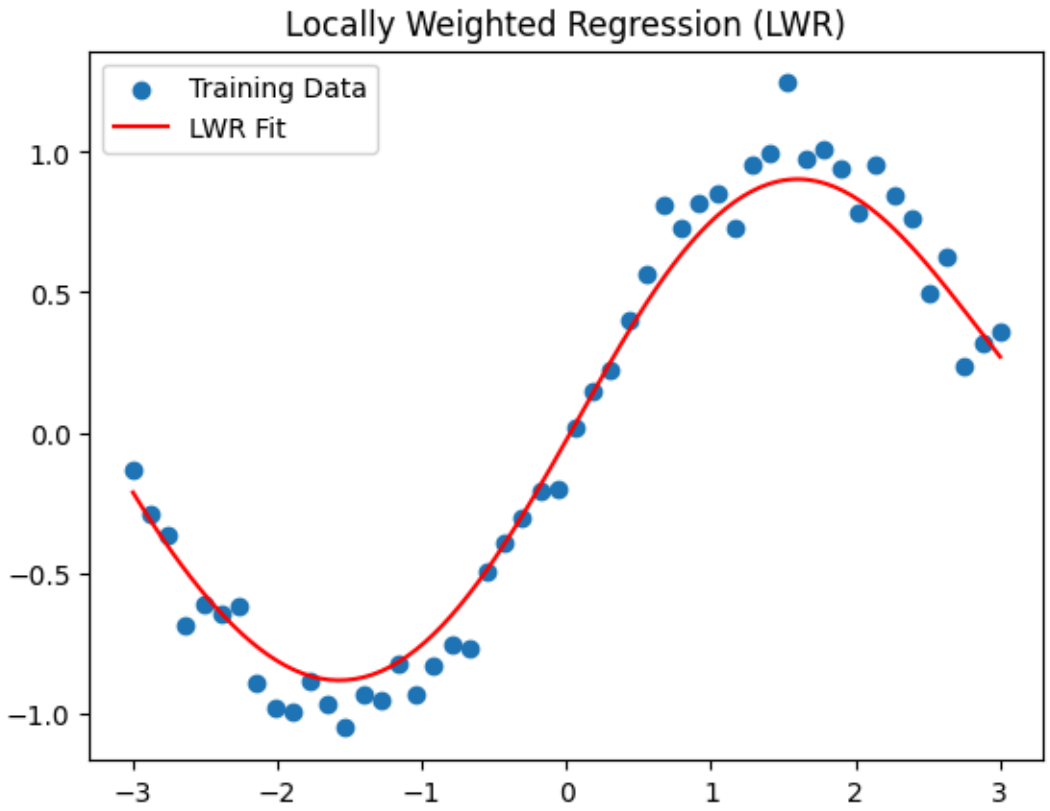
$$O(n \times d)$$

6. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs

```
import numpy as np
import matplotlib.pyplot as plt
def predict(xq, X, y, tau):
    xq = np.array([1, xq])          # add bias term

 # Compute weights using Gaussian kernel
    W = np.diag(np.exp(-np.sum((X - xq)**2, axis=1) / (2 * tau**2)))

 # Compute theta
    theta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ y

    # Predicton
    return xq @ theta

# Generate dataset
x = np.linspace(-3, 3, 50)
y = np.sin(x) + 0.1 * np.random.randn(50)

# Add bias term
X = np.c_[np.ones(x.shape), x]

# Query points
xq = np.linspace(-3, 3, 200)

# Predictions
yp = np.array([predict(xi, X, y, 0.5) for xi in xq])

# Plot
```

```
plt.scatter(x, y, label="Training Data")
plt.plot(xq, yp, 'r', label="LWR Fit")
plt.title('Locally Weighted Regression (LWR)')
plt.legend()
plt.show()
```

OUTPUT



Concept Explanation: Locally Weighted Regression (LWR)

Locally Weighted Regression (LWR), also referred to as Locally Weighted Linear Regression (LWLR), is a non-parametric supervised learning algorithm that performs regression by fitting a model locally around each query point rather than learning a single global function (Hastie et al., 2009).

1. Core Idea

Unlike traditional linear regression, which learns a single global model, LWR:

Builds a separate regression model for every query point

Assigns higher importance (weights) to nearby data points

Assigns lower importance to distant points

Thus, it captures non-linear relationships effectively by approximating them locally.

2. Mathematical Concept

The prediction is based on a **weighted least squares formulation**, where weights are computed using a Gaussian kernel:

Where:

x_q = query point

x_i = training data point

τ= bandwidth parameter

The regression parameters are computed as:

$$\theta = (X^T W X)^{-1} X^T W y$$

3. Working Mechanism

For each query point:

Compute distance from all training points

Assign weights using Gaussian kernel

Construct weight matrix W

Fit a **local linear model**

Predict output

This process is repeated for **every new input**, making LWR a **lazy learning algorithm**.

4. Key Characteristics

Property	**Description**
Learning Type	Lazy learning
Model Type	Non-parametric
Flexibility	High (captures non-linearity)
Computation	Expensive (model per query)
Data Requirement	Stores entire dataset

5. Role of Bandwidth Parameter (τ)

The parameter τcontrols how “local” the model is:

τ Value	Effect
Small τ	Focuses on very nearby points → overfitting
Medium τ	Balanced fitting
Large τ	Behaves like global linear regression → underfitting

6. Advantages

Handles **non-linear data effectively**

No need to assume a global function

Intuitive and interpretable

7. Limitations

Computationally expensive ($O(n^2)$ complexity)

Sensitive to choice of τ

Poor scalability for large datasets

Program Explanation:

1. Importing Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

NumPy → numerical computations (arrays, matrix operations)

Matplotlib → visualization of data and regression curve

2. Prediction Function

def predict(xq, X, y, tau):

This function predicts the output for a **single query point** x_q.

Step 2.1: Add Bias Term

xq = np.array([1, xq])

Converts scalar input into a vector $\left[x_q\right]$

This represents the equation:

$$h(x) = \theta_0 + \theta_1 x$$

Step 2.2: Compute Weights (Gaussian Kernel)

W = np.diag(np.exp(-np.sum((X - xq)**2, axis=1) / (2 * tau**2)))

This is the **core of LWR**.

Weight formula:

$$w_i = exp\left(\frac{(x_i)^2}{2\tau^2}\right)$$

Interpretation:

Points **closer to** x_q → higher weight

Points **far away** → very small weight

Matrix W is a **diagonal matrix of weights**.

Step 2.3: Compute Model Parameters (θ)

theta = np.linalg.pinv(X.T @ W @ X) @ X.T @ W @ y

This is the **weighted least squares solution**:

$$\theta = (X^T W X)^{-1} X^T W y$$

Uses **pseudo-inverse (pinv)** for numerical stability

Computes parameters specific to the query point

Step 2.4: Predict Output

return xq @ theta

- Computes prediction:

$$y_q = \theta_0 + \theta_1 x_q$$

3. Dataset Generation

x = np.linspace(-3, 3, 50)
y = np.sin(x) + 0.1 * np.random.randn(50)

10. Generates **50 points between -3 and 3**

11. Target values:

- True function → $sin(x)$

- Added noise → makes data realistic

4. Add Bias Term

X = np.c_[np.ones(x.shape), x]

Transforms data into:

$$X = [1, x]$$

5. Query Points

xq = np.linspace(-3, 3, 200)

- 200 points used to draw a **smooth prediction curve**

6. Predictions

yp = np.array([predict(xi, X, y, 0.5) for xi in xq])

7. For each query point:
    - Compute weights
    - Fit local model
    - Predict output

This is why LWR is **computationally expensive**:
→ It builds a model for **every point**

7. Plotting the Output

plt.scatter(x, y, label="Training Data")
plt.plot(xq, yp, 'r', label="LWR Fit")

Graph Components:

Element	Meaning
Scatter points	Noisy training data
Red curve	LWR prediction
X-axis	Input feature
Y-axis	Output value

8. Output Interpretation

The graph shows:

1. Training Data

1. Blue dots scattered around the **sine curve**
2. Noise is present

2. LWR Curve (Red Line)

- Smooth curve approximating **sin(x)**
- Adapts locally:
    - Near each point, nearby data dominates
    - Produces **non-linear regression**

9. Effect of Parameter τ(Bandwidth)

τ value	**Behavior**
Small (e.g., 0.1)	Very flexible, overfits noise
Medium (0.5)	Good fit (used here)
Large (1+)	Smooth, may underfit

10. Key Characteristics of LWR

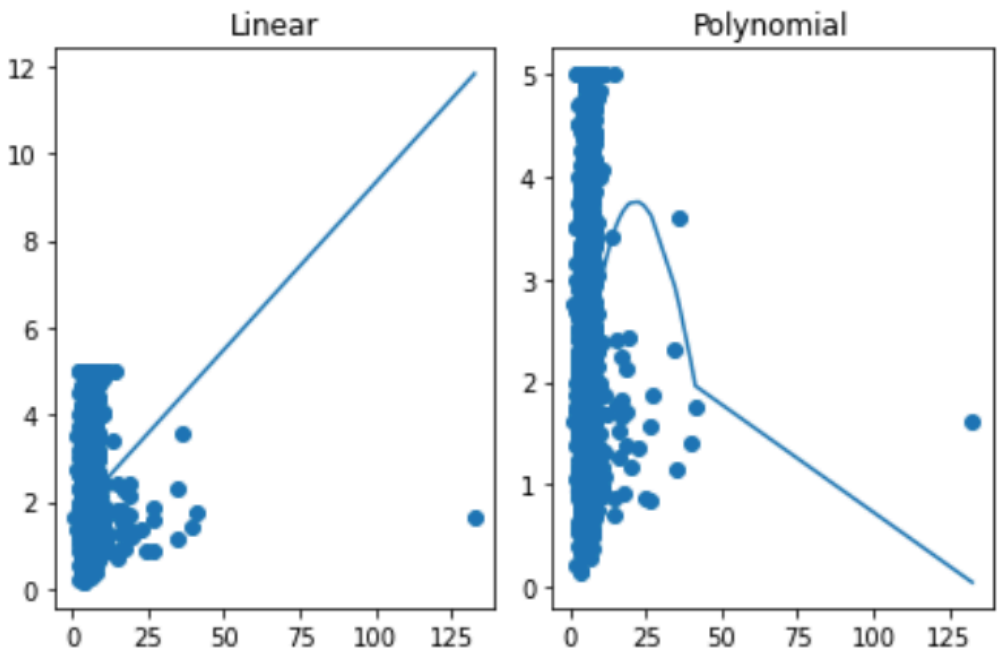
9. **Non-parametric** → no fixed model
10. **Lazy learning** → computation at prediction time
11. **Local fitting** → adapts to nearby data
12. Handles **non-linear patterns effectively**

Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
# Load data
data = fetch_california_housing(as_frame=True)
X = data.data[['AveRooms']]
y = data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Models
lin = LinearRegression().fit(X_train, y_train)
poly = PolynomialFeatures(3)
poly_model = LinearRegression().fit(poly.fit_transform(X_train), y_train)

# Predictions
y_lin = lin.predict(X_test)
y_poly = poly_model.predict(poly.transform(X_test))

# Sort for plotting
x = X_test.values.ravel()
idx = np.argsort(x)

# Plot
```

```
plt.subplot(1,2,1)
plt.scatter(x, y_test)
plt.plot(x[idx], y_lin[idx])
plt.title("Linear")

plt.subplot(1,2,2)
plt.scatter(x, y_test)
plt.plot(x[idx], y_poly[idx])
plt.title("Polynomial")

plt.tight_layout()
plt.show()

# MSE
print("Linear MSE:", mean_squared_error(y_test, y_lin))
print("Polynomial MSE:", mean_squared_error(y_test, y_poly))
```

OUTPUT:




```
Linear MSE: 1.2923314440807299
Polynomial MSE: 1.2117194623741487
```

<u>Program Explanation:</u>

import numpy as np

Imports **NumPy** for numerical operations (arrays, sorting, etc.)

import matplotlib.pyplot as plt

Imports **Matplotlib** to create graphs and plots

from sklearn.linear_model import LinearRegression

Imports **Linear Regression model** from Scikit-learn

from sklearn.preprocessing import PolynomialFeatures

Used to **convert input features into polynomial form** (e.g., $x \to x^2, x^3$)

from sklearn.metrics import mean_squared_error

Used to calculate **error between actual and predicted values**

from sklearn.model_selection import train_test_split

Splits dataset into **training and testing sets**

from sklearn.datasets import fetch_california_housing

Loads **California Housing dataset**

- **Load Dataset**

data = fetch_california_housing(as_frame=True)

- Loads dataset as a **Pandas DataFrame**

X = data.data[['AveRooms']]

12. Selects only **'AveRooms' (average number of rooms)** as input feature

y = data.target

- Target variable = **house price**

🔹 **Split Data**

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

8. Splits data into:
    - **80% training**
    - **20% testing**
9. random_state=42 ensures **same split every time**

🔹 **Train Models**

Linear Regression

lin = LinearRegression().fit(X_train, y_train)

3. Creates and trains a **linear model**
4. Learns equation:
   👉 $y = mx + b$

Polynomial Regression

poly = PolynomialFeatures(3)

- Converts input into **degree 3 polynomial**
- Example:
    - $x \rightarrow x, x^2, x^3$

poly_model = LinearRegression().fit(poly.fit_transform(X_train), y_train)

13. Transforms training data into polynomial features
14. Then trains **linear regression on transformed data**
15. Final model becomes:
    👉 $y = a + bx + cx^2 + dx^3$

🔹 **Predictions**

y_lin = lin.predict(X_test)

7. Predicts values using **linear model**

y_poly = poly_model.predict(poly.transform(X_test))

4. Converts test data to polynomial form
5. Predicts using **polynomial model**

🔹 **Sorting for Plotting**

x = X_test.values.ravel()

- Converts test feature into **1D array**

idx = np.argsort(x)

- Gets indices that **sort x values**
- Needed so line plot appears smooth (not zig-zag)

🔹 **Plot Graphs**

Linear Plot

plt.subplot(1,2,1)

6. Creates subplot (1 row, 2 columns, position 1)

plt.scatter(x, y_test)

- Plots **actual data points**

plt.plot(x[idx], y_lin[idx])

7. Plots **linear prediction line**

plt.title("Linear")

3. Title of plot

Polynomial Plot

plt.subplot(1,2,2)

- Second subplot

plt.scatter(x, y_test)

10. Actual data points

plt.plot(x[idx], y_poly[idx])

14. Polynomial curve (non-linear)

plt.title("Polynomial")

9. Title

🔹 **Layout & Show**

plt.tight_layout()

- Adjusts spacing between plots

plt.show()

- Displays plots

🔹 **Error Calculation**

print("Linear MSE:", mean_squared_error(y_test, y_lin))

7. Calculates error for **linear model**

print("Polynomial MSE:", mean_squared_error(y_test, y_poly))

- Calculates error for **polynomial model**

🔹 **Final Concept Summary**

1. **Linear Regression** → Straight line fit
2. **Polynomial Regression** → Curved fit
3. **MSE (Mean Squared Error)**:
    1. Lower value = **better model**

Concept Explanation

Linear Regression

Linear Regression tries to model a **straight-line relationship** between input X and output y.

Equation:

3. $b_0 \rightarrow$ intercept
4. $b_1 \rightarrow$ slope
5. Assumes **linear relationship**

Example:

1. Predict house price using number of rooms (Boston dataset)

Polynomial Regression

Polynomial Regression is used when the relationship is **curved (non-linear)**.

Equation:

- Converts input into higher-degree terms
- Still uses linear model internally but on transformed features

Example:

- Predict fuel efficiency (MPG) based on horsepower (Auto MPG dataset)

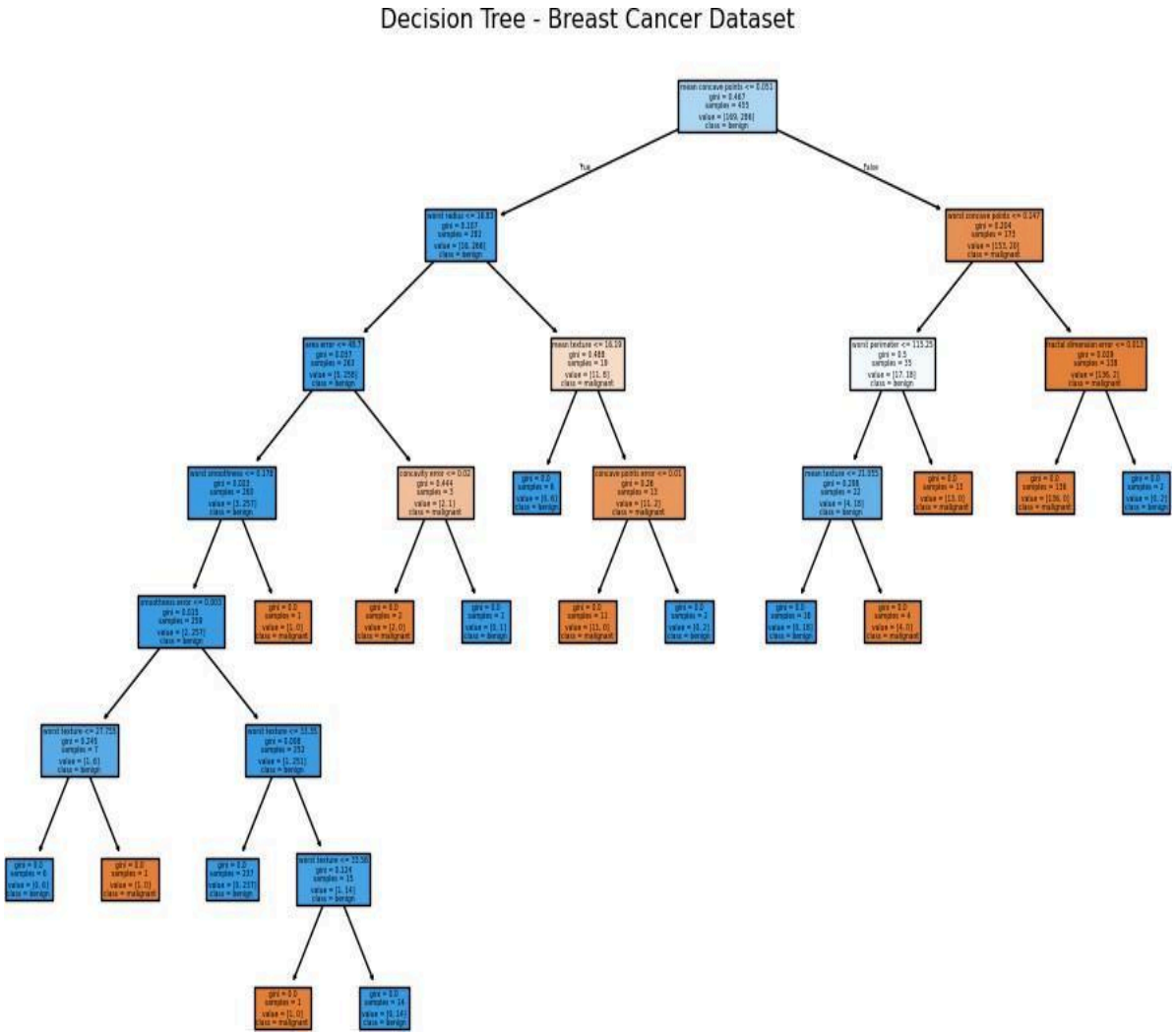
Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

```
# Importing necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
x, y = data.data, data.target

# Split data
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)

# Train model
model = DecisionTreeClassifier()
model.fit(x_train, y_train)

# Predict and accuracy
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Predict one sample
pred = model.predict([x_test[0]])
print("Prediction:", "Benign" if pred == 1 else "Malignant")

# Plot tree
plt.figure(figsize=(10,6))
plot_tree(model, filled=True)
plt.show()
```

OUTPUT:

Model Accuracy: 94.74%

Predicted Class for the new sample: Benign



Program Explanation

1. Import Libraries

import numpy as np

Imports **NumPy**, a library used for **numerical operations and arrays**.

In this program it is not heavily used, but usually helps with data handling.

import matplotlib.pyplot as plt

Imports **Matplotlib** for **data visualization and plotting graphs**.

plt is the short name used to call plotting functions.

from sklearn.datasets import load_breast_cancer

Imports the **Breast Cancer dataset** from **Scikit-learn**.

This dataset contains features of tumors used to classify them as **benign or malignant**.

from sklearn.model_selection import train_test_split

Imports a function used to **split the dataset into training and testing sets**.

from sklearn.tree import DecisionTreeClassifier, plot_tree

DecisionTreeClassifier → algorithm used to **build a decision tree model**.

plot_tree → used to **visualize the trained decision tree**.

from sklearn.metrics import accuracy_score

Imports a function to **measure the model accuracy**.

---

2. Load Dataset

data = load_breast_cancer()

Loads the **breast cancer dataset** into the variable data.

The dataset contains:

- **Features (X)** → measurements of tumors
- **Target (y)** → cancer type (benign or malignant)

---

3. Separate Features and Target

x, y = data.data, data.target

13. data.data → input features (tumor measurements)
14. data.target → output label (0 or 1)

So:

- **x → input variables**
- **y → class labels**

Example:

Feature1	Feature2	Feature3	...	Target
14.2	20.3	90.1	...	0
12.1	15.4	70.5	...	1

---

4. Split Dataset

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)

This splits the dataset into **training and testing data**.

10. **80% → Training data**
11. **20% → Testing data**

Variables created:

Variable	Meaning
x_train	training features
x_test	testing features
y_train	training labels
y_test	testing labels

Purpose:

5. Train the model on training data
6. Test the model on unseen data

---

5. Create Decision Tree Model

model = DecisionTreeClassifier()

- Creates a **Decision Tree classification model**.
- The tree will learn rules to classify tumors.

Example rule:

If radius > 15 → Malignant
Else → Benign

---

6. Train the Model

model.fit(x_train, y_train)

16. Trains the decision tree using **training data**.

Meaning:

Input : x_train
Output: y_train

The model learns patterns from the data.

---

7. Predict Test Data

y_pred = model.predict(x_test)

8. Uses the trained model to **predict labels for test data**.

So:

x_test → model → predicted result (y_pred)

---

8. Calculate Accuracy

print("Accuracy:", accuracy_score(y_test, y_pred))

6. Compares **actual values (y_test)** with **predicted values (y_pred)**.

Formula for accuracy:

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$$

Example:

Actual	**Predicted**
Benign	Benign
Malignant	Malignant
Benign	Malignant

If 8 correct out of 10:

Accuracy = 8/10 = 0.8 = 80%

---

9. Predict One Sample

pred = model.predict([x_test[0]])

- Predicts the class for **one sample** from test data.

x_test[0] → first sample of test dataset.

[ ] is used because the model expects **2D input**.

---

10. Print Prediction

print("Prediction:", "Benign" if pred == 1 else "Malignant")

This is a **conditional statement**.

Meaning:

If pred == 1 → print Benign
Else → print Malignant

Because dataset labels:

Value	Meaning
0	Malignant
1	Benign

---

11. Create Plot Figure

plt.figure(figsize=(10,6))

Creates a **figure window** for plotting.

- Width = 10
- Height = 6

This improves visualization.

---

12. Plot the Decision Tree

plot_tree(model, filled=True)

Displays the **decision tree structure**.

Each node shows:

7. Feature used
8. Threshold value
9. Class prediction

filled=True → colors the nodes based on class.

Example visualization:

```
radius ≤ 14
  /      \
Benign   Malignant
```

---

13. Show the Plot

plt.show()

Displays the tree graph on screen.

Final Flow of the Program

```
Load dataset
    ↓
Split into training and testing data
    ↓
Train Decision Tree model
    ↓
Predict test data
    ↓
Calculate accuracy
    ↓
Predict one sample
    ↓
Display decision tree
```

Concept of Decision Tree

A **Decision Tree** is a supervised learning algorithm used for **classification and regression**.

It works like a **flowchart**:

- Each **internal node** → a feature (condition)
- Each **branch** → decision outcome
- Each **leaf node** → final class label

How it Works

8. Select the **best feature** to split the data
9. Divide dataset into subsets

10. Repeat recursively until:
    a. Data is pure OR
    b. Maximum depth reached

Splitting Criteria

Decision Trees use measures like:

🔹 **Gini Index**

4. Measures impurity
5. Lower value → better split

🔹 **Entropy (Information Gain)**

- Measures randomness
- Used to maximize information gain

Example (Breast Cancer Dataset)

11. Features: tumor size, texture, radius, etc.
12. Output:
    a. **0 → Malignant (Cancerous)**
    b. **1 → Benign (Non-cancerous)**

9. Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

```
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt

# Load + split
X, y = fetch_olivetti_faces(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42)

# Train + predict
model = GaussianNB()
pred = model.fit(Xtr, ytr).predict(Xte)

# Evaluation
print("Accuracy:", accuracy_score(yte, pred))
print("\nReport:\n", classification_report(yte, pred, zero_division=1))
print("\nConfusion Matrix:\n", confusion_matrix(yte, pred))
print("\nCV Accuracy:", cross_val_score(model, X, y, cv=5).mean())

# Display images
fig, ax = plt.subplots(3, 5, figsize=(10, 6))
for a, img, t, p in zip(ax.ravel(), Xte, yte, pred):
    a.imshow(img.reshape(64,64), cmap='gray')
    a.set_title(f"T:{t} P:{p}")
    a.axis('off')
plt.tight_layout(); plt.show()
```

OUTPUT:

Accuracy: 0.7416666666666667

Report:

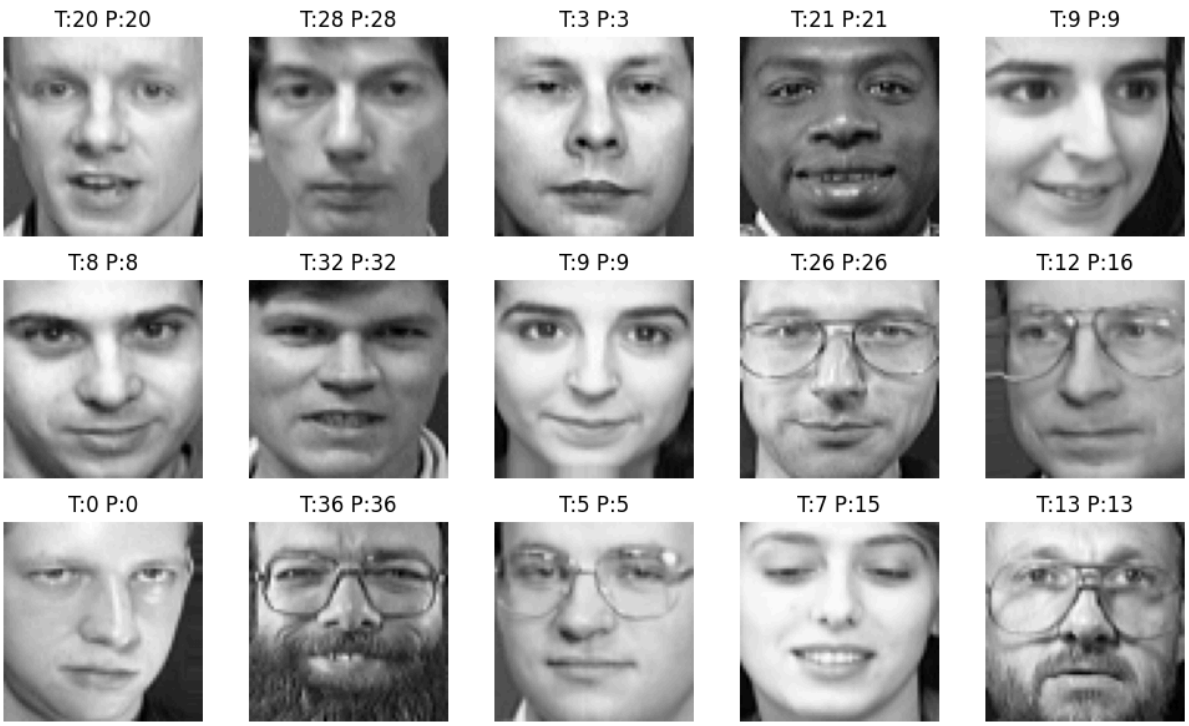
	precision	recall	f1-score	support
0	1.00	0.60	0.75	5
1	1.00	0.75	0.86	4
2	1.00	1.00	1.00	2
3	1.00	1.00	1.00	4
4	1.00	0.67	0.80	3
5	1.00	1.00	1.00	3
6	1.00	0.33	0.50	3
7	1.00	0.00	0.00	8
8	1.00	1.00	1.00	2
9	0.30	1.00	0.46	3
10	1.00	0.67	0.80	3
11	1.00	0.80	0.89	5
12	0.25	0.50	0.33	2
13	1.00	0.67	0.80	3
14	1.00	1.00	1.00	3
15	0.50	1.00	0.67	3
16	0.00	1.00	0.00	0
17	1.00	0.33	0.50	3
18	1.00	0.50	0.67	2
19	1.00	1.00	1.00	1
20	1.00	1.00	1.00	2
21	1.00	1.00	1.00	1
22	1.00	1.00	1.00	4
23	1.00	1.00	1.00	4
24	1.00	0.67	0.80	3
25	0.50	0.50	0.50	2
26	1.00	0.75	0.86	4

27	1.00	1.00	1.00	3
28	0.36	1.00	0.53	4
29	1.00	1.00	1.00	3
30	1.00	1.00	1.00	2
31	1.00	1.00	1.00	1
32	1.00	1.00	1.00	3
33	1.00	1.00	1.00	2
34	0.67	1.00	0.80	2
35	0.50	1.00	0.67	2
36	1.00	1.00	1.00	2
37	1.00	1.00	1.00	3
38	1.00	0.00	0.00	7
39	0.57	1.00	0.73	4
accuracy			0.74	120
macro avg	0.87	0.82	0.77	120
weighted avg	0.90	0.74	0.73	120

Confusion Matrix:

```
[[3 0 0 ... 0 0 0]
 [0 3 0 ... 0 0 0]
 [0 0 2 ... 0 0 0]
 ...
 [0 0 0 ... 3 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 4]]
```

CV Accuracy: 0.8825



Explanation:

1. Import Libraries

from sklearn.datasets import fetch_olivetti_faces

Loads **face dataset**

from sklearn.model_selection import train_test_split, cross_val_score

- train_test_split → splits data
- cross_val_score → evaluates model multiple times

from sklearn.naive_bayes import GaussianNB

Machine learning model (Naive Bayes)

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

Used to evaluate model performance

import matplotlib.pyplot as plt

Used to display images

2. Load Dataset

X, y = fetch_olivetti_faces(return_X_y=True)

Dataset details:

- X → image data (features)
- y → labels (person IDs)

Each image:

- Size = **64 × 64 pixels**
- Flattened → **4096 features**

3. Split Dataset

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42)

Splits data into:

3. 70% → training set
4. 30% → testing set

Why?

- Train model on one part
- Test on unseen data

4. Train Model

model = GaussianNB()

Creates Naive Bayes classifier

pred = model.fit(Xtr, ytr).predict(Xte)

Two steps:

Fit:

1. Learns from training data

Predict:

3. Predicts labels for test data

5. Accuracy Calculation

print("Accuracy:", accuracy_score(yte, pred))

Formula:

$$Accuracy = \frac{Correct\ predictions}{Total\ predictions}$$

Example output:

Accuracy: 0.82

6. Classification Report

classification_report(yte, pred, zero_division=1)

Displays:

- Precision
- Recall
- F1-score

For each class (person)

zero_division=1:

- Avoids warning when values undefined

7. Confusion Matrix

confusion_matrix(yte, pred)

Table showing:

- Rows → actual labels
- Columns → predicted labels

Helps identify:

- Correct predictions
- Misclassifications

8. Cross Validation

cross_val_score(model, X, y, cv=5).mean()

Process:

- Split data into 5 parts
- Train/test 5 times

Gives **more reliable accuracy**

9. Create Image Grid

fig, ax = plt.subplots(3, 5, figsize=(10, 6))

Creates:

3 rows × 5 columns

Total = **15 images**

10. Loop Through Images

for a, img, t, p in zip(ax.ravel(), Xte, yte, pred):

Iterates through:

- img → image
- t → true label

- p → predicted label

Display Image

a.imshow(img.reshape(64,64), cmap='gray')

Converts:

- 1D vector → 64×64 image

Add Title

a.set_title(f"T:{t} P:{p}")

Shows:

7. T = actual class
8. P = predicted class

Remove Axis

a.axis('off')

Cleaner display

11. Show Output

plt.tight_layout()
plt.show()

Displays all images properly

Final Output

You will get:

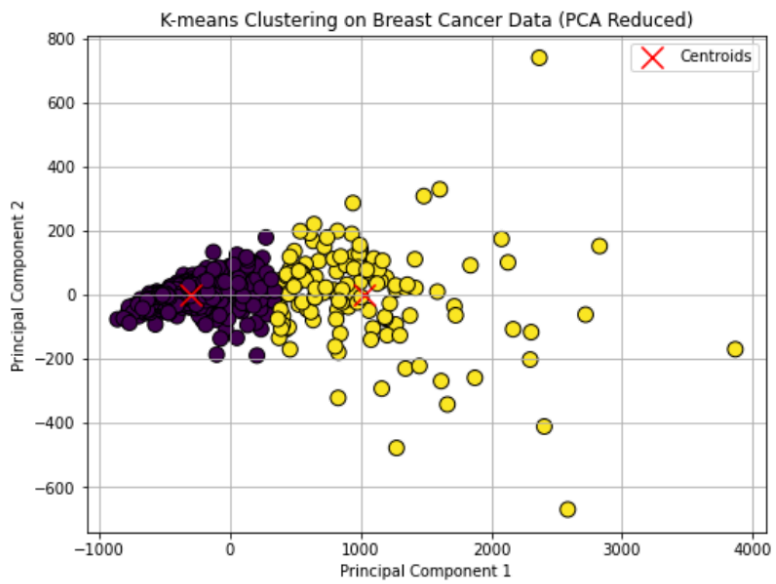
✔ Accuracy (~80–90%)
✔ Classification report
✔ Confusion matrix
✔ Cross-validation accuracy
✔ 15 face images with labels

10. Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans


# Step 1: Load the Wisconsin Breast Cancer dataset

data = load_breast_cancer()

X = data.data        # Features

y = data.target      # Labels (0 = malignant, 1 = benign)

# Step 2: Apply PCA for dimensionality reduction
# (from 30 features to 2 features for visualization)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)


# Step 3: Perform K-means clustering
# 2 clusters because dataset has 2 classes

kmeans = KMeans(n_clusters=2, random_state=42)

kmeans.fit(X_pca)


# Step 4: Visualize the clustering results
plt.figure(figsize=(8, 6))

plt.scatter(

    X_pca[:, 0],

    X_pca[:, 1],

    c=kmeans.labels_,

    cmap='viridis',

    marker='o',

    s=100,

    edgecolor='k')
```

```
plt.title("K-means Clustering on Breast Cancer Data (PCA Reduced)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

# Show cluster centers
centers = kmeans.cluster_centers_
plt.scatter(
    centers[:, 0],
    centers[:, 1],
    s=200,
    c='red',
    marker='x',
    label='Centroids'
)
# Add legend and grid
plt.legend()
plt.grid(True)
# Display plot
plt.show()
```

OUTPUT:



Program Explanation:

import numpy as np

Imports NumPy for numerical operations (arrays, math).

import pandas as pd

Imports Pandas for handling datasets (not heavily used here).

import matplotlib.pyplot as plt

Used to plot graphs and visualize data.

from sklearn.datasets import load_breast_cancer

Loads the Breast Cancer dataset from sklearn.

from sklearn.decomposition import PCA

Imports PCA (Principal Component Analysis) for reducing dimensions.

from sklearn.cluster import KMeans

Imports K-Means algorithm for clustering.

Step 1: Load Dataset

data = load_breast_cancer()

Loads dataset into a variable data.

X = data.data

X contains input features (30 features per sample).

y = data.target

y contains labels:

0 → malignant

1 → benign

Step 2: Apply PCA

pca = PCA(n_components=2)

- Creates a PCA model to reduce data to 2 dimensions.

X_pca = pca.fit_transform(X)

15. Applies PCA:

- Fits model to data
- Transforms data into 2D (for visualization)

Step 3: K-Means Clustering

```
kmeans = KMeans(n_clusters=2, random_state=42)
```

- Creates K-Means model:
    - n_clusters=2 → 2 groups
    - random_state=42 → same results every run

```
kmeans.fit(X_pca)
```

12. Fits K-Means to the PCA-reduced data.

Step 4: Plot the Results

```
plt.figure(figsize=(8, 6))
```

7. Creates a plot of size 8×6.

```
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=kmeans.labels_,
    cmap='viridis',
    marker='o',
    s=100,
    edgecolor='k')
```

- Plots data points:
    - X_pca[:, 0] → X-axis (PC1)
    - X_pca[:, 1] → Y-axis (PC2)
    - c=kmeans.labels_ → colors based on cluster
    - cmap='viridis' → color style
    - marker='o' → circle points
    - s=100 → size of points
    - edgecolor='k' → black border

Add Labels and Title

```
plt.title("K-means Clustering on Breast Cancer Data (PCA Reduced)")
```

17. Adds title to graph.

```
plt.xlabel("Principal Component 1")
```

9. Label for X-axis.

```
plt.ylabel("Principal Component 2")
```

7. Label for Y-axis.

Plot Cluster Centers

centers = kmeans.cluster_centers_

- Gets the centroids (center points) of clusters.

```
plt.scatter(
    centers[:, 0],
    centers[:, 1],
    s=200,
    c='red',
    marker='x',
    label='Centroids'
)
```

- Plots centroids:
    - Larger size (s=200)
    - Red color
    - Marker x

Final Touches

plt.legend()

10. Shows legend (centroids label).

plt.grid(True)

- Adds grid lines.

plt.show()

11. Displays the final plot.

Concept of K-Means Clustering

K-Means is an **unsupervised learning algorithm** used to group data into **K clusters** based on similarity.

It does NOT use labels (unlike classification)

How K-Means Works

6. Choose number of clusters K
7. Initialize Kcentroids randomly
8. Assign each point to the **nearest centroid**

9. Recompute centroids (mean of cluster points)
10. Repeat until centroids stop changing

Objective Function

K-Means minimizes the distance between points and their cluster centers:

- C_i → cluster
- μ_i → centroid
- Goal → minimize total squared distance

Dataset (Breast Cancer)

13. Features: tumor characteristics
14. Labels exist (benign/malignant), but K-Means **ignores them**
15. We use them only to compare clustering quality